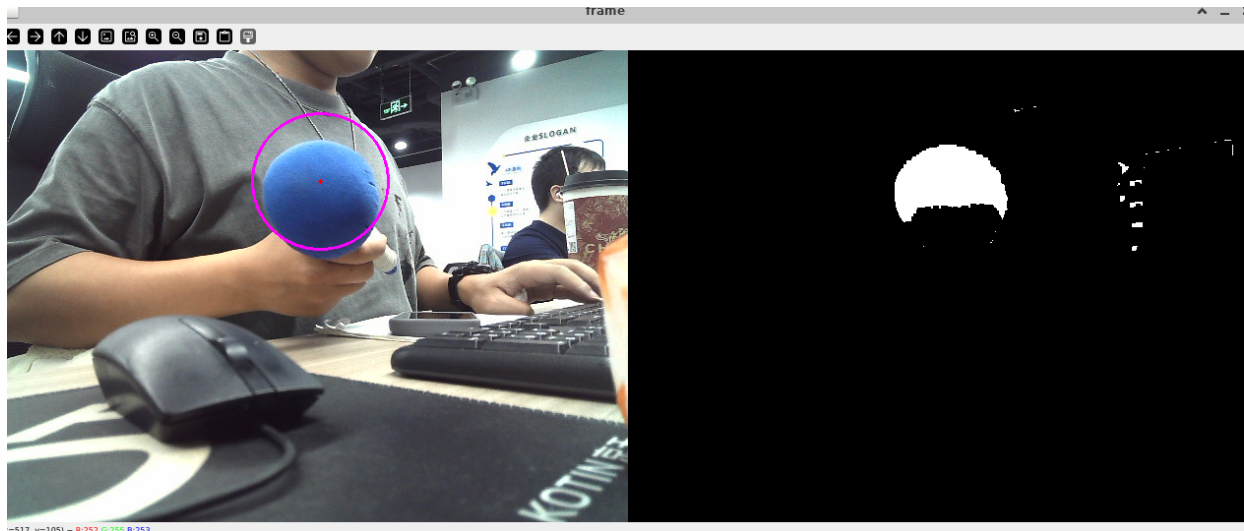


3.Color tracking

1. Program startup

Input following command in terminal

```
ros2 launch astra_camera gemini.launch.xml  
ros2 run yahboomcar_astra colorHSV  
ros2 run yahboomcar_astra colorTracker
```



After successful startup, the above screen will be displayed. The program will initially load the HSV values and then display the processed image.

Press the [r] key to reselect the color, and use the mouse to mark the color to be tracked. The selected area can only have one color.

After selecting the color to be tracked, the program processes the image and presses the spacebar to start tracking.

The terminal that starts colorTracker will display the following interface.

```

sunrise@ubuntu:~$ ros2 run yahboomcar_astra colorTracker
import done
init done
start it
twist.linear.x: 0.0
twist.angular.z: 0.8999999999999999
twist.linear.x: 0.0
twist.angular.z: 0.18
twist.linear.x: 0.0
twist.angular.z: 0.18
twist.linear.x: 0.0
twist.angular.z: 0.18
twist.linear.x: 0.0
twist.angular.z: 0.18
twist.linear.x: 0.0
twist.angular.z: 0.18
twist.linear.x: 0.0
twist.angular.z: 0.15500000000000003
twist.linear.x: 0.0
twist.angular.z: 0.17500000000000002

```

这里原本是要结合机器人来运行的，没有驱动底盘的程序就只能通过打印需要发布的速度来验证，移动选中的物体，打印出来的线速度和角速度会变化，这里的速度话题是/cmd_vel，有机器人底盘驱动程序订阅了这个节点的，即可驱动机器人。

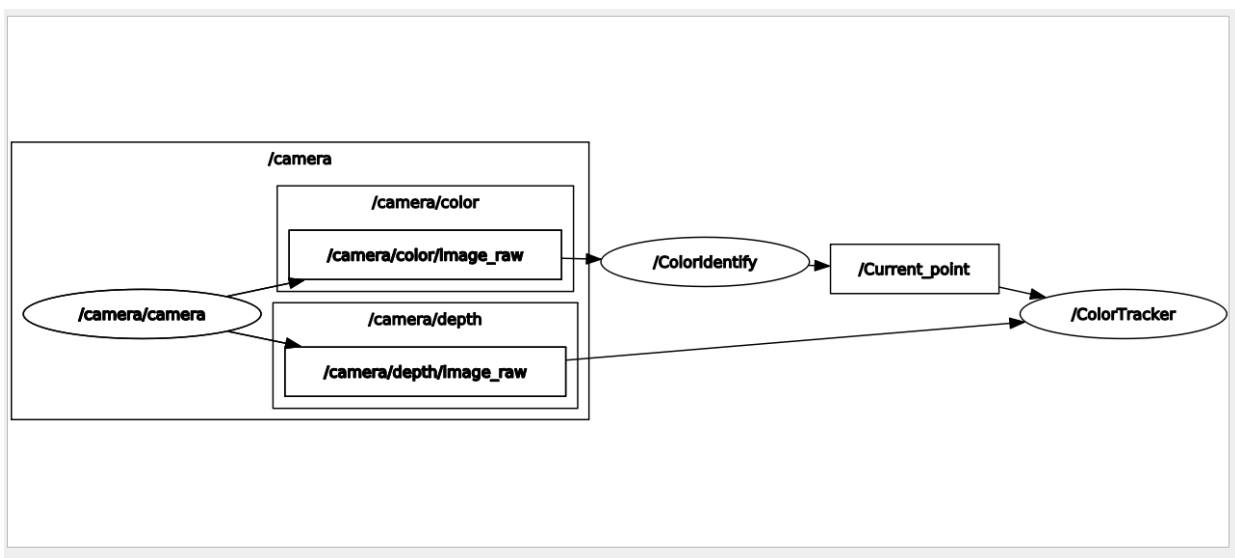
查看节点间的通讯，终端输入，

This was originally intended to run in conjunction with robots, without a program to drive the chassis, it could only be verified by printing the required speed to move the selected object.

The printed linear velocity and angular velocity will change. The speed topic here is/cmd_vel. If a robot chassis driver program subscribes to this node, it can drive the robot.

Input following command in terminal to view communication between nodes.

```
ros2 run rqt_graph rqt_graph
```



2. Core code analysis

Code path:

```
~/orbbec_ws/src/yahboomcar_astra/yahboomcar_astra/colorHSV.py  
~/orbbec_ws/src/yahboomcar_astra/yahboomcar_astra/colorTracker.py
```

3.2.1、colorHSV.py

This program mainly has the following functions:

- Subscribe to camera image data;
- Obtain keyboard and mouse events for switching modes and color selection;
- Process images and publish the center coordinates of tracked objects and publish them;

Some of the core code is as follows,

```
#Create publishers and subscribers  
self.pub_position = self.create_publisher(Position, "/Current_point", 10)  
self.sub_img  
=self.create_subscription(Image, '/camera/color/image_raw', self.handleTopic, 1)  
#Subscribe to the image callback function and pass the image to the process function  
frame, binary = self.process(frame, action)  
#Retrieve keyboard and mouse events to obtain the value of HSV;  
if action == 32: self.Track_state = 'tracking'  
elif action == ord('i') or action == ord('I'): self.Track_state =  
"identify"  
elif action == ord('r') or action == ord('R'): self.Reset()  
elif action == ord('q') or action == ord('Q'): self.cancel()  
if self.Track_state == 'init':  
cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)  
cv.setMouseCallback(self.windows_name, self.onMouse, 0)  
if self.select_flags == True:  
cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)  
cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)  
if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=  
self.Roi_init[3]:  
rgb_img, self.hsv_range = self.color.Roi_hsv(rgb_img,  
self.Roi_init)  
self.gTracker_state = True  
self.dyn_update = True  
else: self.Track_state = 'init'  
#Calculate the value of the center coordinate, and store the xy value in self.circle  
rgb_img, binary, self.circle = self.color.object_follow(rgb_img, self.hsv_range)  
#Publish a message about the center coordinates  
threading.Thread(target=self.execute, args=(self.circle[0], self.circle[1],  
self.circle[2])).start()  
def execute(self, x, y, z):  
position = Position()  
position.angle_x = x * 1.0  
position.angle_y = y * 1.0
```

```
position.distance = z * 1.0
self.pub_position.publish(position)
```

3.2.2、colorTracker.py

This program mainly has the following functions: receiving/Current_point and depth image topic data, calculating speed size, and then publishing speed data.

```
#Define subscribers to receive the required topic data
self.sub_depth =
self.create_subscription(Image, "/camera/depth/image_raw", self.depth_img_Callback
, 1)
self.sub_position
=self.create_subscription(Position, "/Current_point", self.positionCallback, 1)
#Define speed publisher
self.pub_cmdVel = self.create_publisher(Twist, '/cmd_vel', 10)
#Two important callback functions, obtaining the self. Center-x value and distance_
value
def positionCallback(self, msg):
def depth_img_Callback(self, msg):
    #The self. Center-x value and distance_ value are calculated based on the linear
velocity and angular velocity
    self.execute(self.Center_x, distance_)
    def execute(self, point_x, dist):
        self.get_param()
        if abs(self.prev_dist - dist) > 300:
            self.prev_dist = dist
        return
        if abs(self.prev_angular - point_x) > 300:
            self.prev_angular = point_x
        return
    if self.Joy_active == True: return
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(320, point_x)
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    if angular_z > 2.0:
        angular_z = 2.0
    if angular_z < -2.0:
        angular_z = -2.0
    if linear_x > 1.0:
        linear_x = 1.0
    if linear_x < -1.0:
        linear_x = -1.0
    twist.angular.z = angular_z * 1.0
    twist.linear.x = linear_x * 1.0
```